

Data Driven Solutions and Discoveries in Mechanics

Using Physics Informed Neural Networks

ES Sukruth

1. Problem Statement

Many mechanics phenomena are governed by differential equations derived from conservation laws. Classical solvers such as the finite element method need a mesh and cannot easily use measured data. This project uses **Physics-Informed Neural Networks (PINNs)**, which embed the governing equation in the network's loss through automatic differentiation, to solve two classes of problems: **data-driven solutions** (forward problems: predict the field from the physics) and **data-driven discoveries** (inverse problems: identify unknown physical parameters from sparse, noisy measurements). We study (A) a vibrating string, governed by the 1D wave equation $u_{tt} = c^2 u_{xx}$, with a standard PINN, and (B) anomalous diffusion in materials with memory, governed by the time-fractional diffusion equation $\partial^\alpha u / \partial t^\alpha = u_{xx} + f$ with a Caputo derivative ($0 < \alpha < 1$), using a **fractional PINN (fPINN)**. For $\alpha = 1$, Part B reduces to the classical consolidation equation studied in the reference paper (Zhang, Chen & Yang).

2. Dataset

A PINN needs no labelled training set; its "data" are points where the physics is enforced. All data are generated synthetically from known exact solutions, which also serve as ground truth.

- **Part A ($x, t \in [0,1]$, $c = 1$):** exact solution $u = \sin(\pi x)\cos(\pi t) + 0.5 \sin(2\pi x)\cos(2\pi t)$. We use 5,000 interior collocation points (Latin Hypercube Sampling), 400 boundary points ($u = 0$ at $x = 0, 1$) and 200 initial points (u and u_t given). For the inverse problem: 500 measurements of u with 1% Gaussian noise; the noise study uses 0–10% noise.
- **Part B ($\alpha = 0.5$):** manufactured solution $u = t^2 \sin(\pi x)$ with the matching source term f . Structured grid of 100 x -positions $\times$ 101 time levels (10,100 points), as required by the fractional operator. For the inverse problem: 300 measurements with 1% noise.
- **Test set:** a separate 201×201 grid (40,401 points) not used in training.

3. Approach

A fully connected network $\hat{u}(x, t; \theta)$ is trained to minimise a physics-informed loss:

$$L(\theta, \gamma) = w_f L_{PDE} + w_b L_{BC} + w_i L_{IC} + w_d L_{data}$$

Each term is a mean squared error. Derivatives such as u_{xx} and u_{tt} are computed exactly with automatic differentiation. In inverse problems, the unknown parameter γ (c or α) is a trainable variable optimised together with θ . Optimisation uses Adam followed by L-BFGS: Adam explores quickly, and L-BFGS refines the solution using curvature information.

fPINN: autograd cannot compute fractional derivatives, which depend on the whole time history. Following Pang et al. (2019), the Caputo derivative is discretised with the **L1 scheme**, written as a matrix product $D^\alpha U = U M^T$, while u_{xx} still comes from autograd. We first verified this operator against an analytical Caputo derivative: it converges at rate 1.49, matching the theoretical $2 - \alpha = 1.5$. The initial and boundary conditions are imposed as **hard constraints**, $\hat{u} = t \cdot x(1-x) \cdot N(x, t)$, the improvement the reference paper suggested for its boundary errors. For the inverse problem, the L1 operator is rewritten to be differentiable in α , with $\alpha = \text{sigmoid}(a)$ to keep $0 < \alpha < 1$.

4. Implementation

- **Framework:** PyTorch 2.11 on Google Colab (Tesla T4 GPU); NumPy, SciPy (qmc sampling, Γ function), Matplotlib.
- **Network:** 4 hidden layers $\times$ 64 neurons, tanh activation (smooth, with non-zero second derivatives, unlike ReLU), Xavier initialisation, inputs scaled to $[-1, 1]$; 12,737 parameters.
- **Training:** Adam for 10,000 iterations (learning rate 10^{-3} , halved every 5,000), then L-BFGS (strong-Wolfe line search). Loss weights for Part A: $w_f = 1$, $w_b = w_i = w_d = 10$. Each run takes about 2–4 minutes on the GPU.
- **Evaluation:** relative L2 error, maximum absolute error, PDE residual on the test grid, and the error in the recovered parameter.

5. Results

Exp.	Method	Task	Result
A1	PINN	Wave equation – forward	Rel. L2 = 1.23×10^{-3} ; PDE loss: train 1.39×10^{-5} , test 1.66×10^{-5}
A2	PINN	Discover c (1% noise, guess 0.5)	$c = 1.00021$ (0.021% error); rel. L2 (u) = 2.17×10^{-3}
A3	PINN	Noise study (0, 1, 5, 10%)	c error < 0.011% at all levels; rel. L2 (u) rises $2.8 \times 10^{-3} \rightarrow 5.2 \times 10^{-3}$
B1	fPINN	Fractional diffusion – forward	Rel. L2 = 1.26×10^{-4} ; max error 1.13×10^{-4}
B2	fPINN	Discover α (1% noise, guess 0.8)	$\alpha = 0.50332$ (0.66% error); rel. L2 (u) = 4.17×10^{-4}

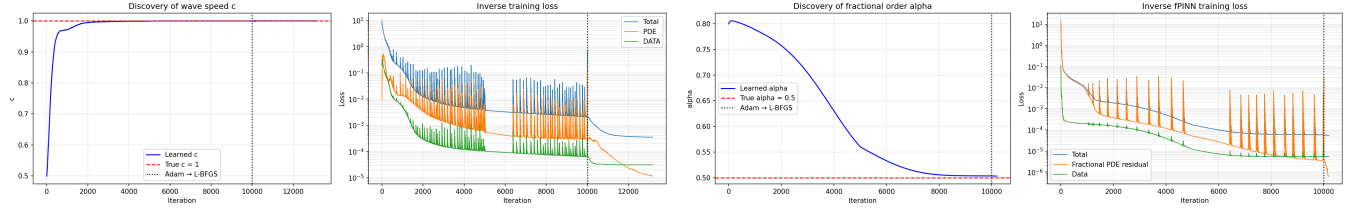


Figure 1. Parameter discovery: wave speed c (left) and fractional order α (right) converge to their true values during training.

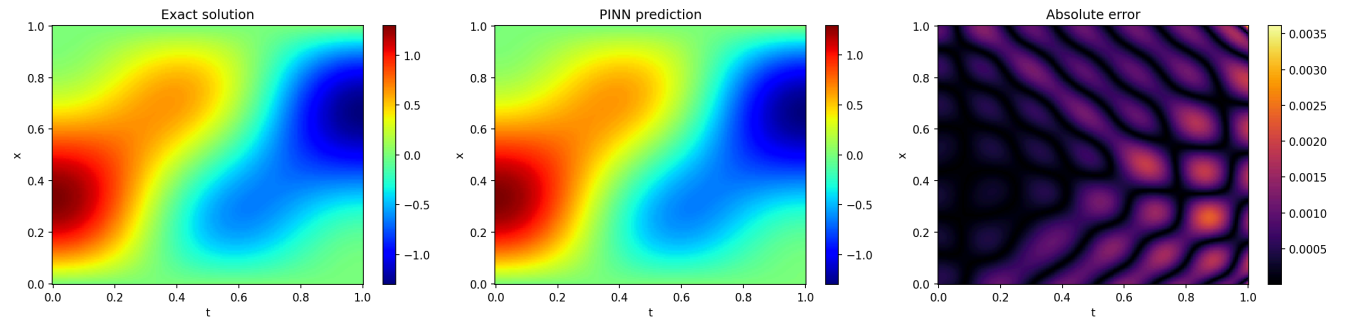


Figure 2. Part A1 – exact solution, PINN prediction and absolute error for the wave equation.

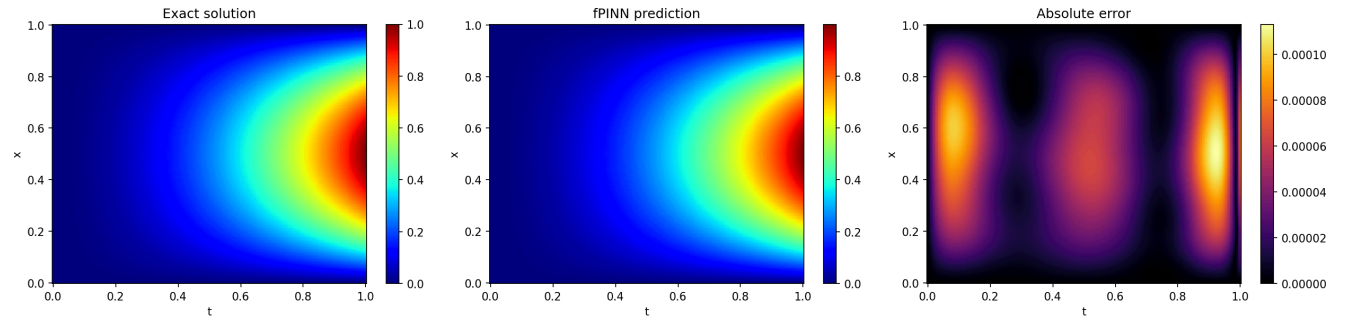


Figure 3. Part B1 – exact solution, fPINN prediction and absolute error; the error is exactly zero on the boundaries (hard constraints).

The PDE residual is almost the same on the test points as on the training points, so the network satisfies the physics everywhere, not only where it was trained. In both inverse problems, the data loss levels off at the noise variance: the physics stops the network from overfitting the noise. The fPINN is about $10\times$ more accurate than the standard PINN, and its error is exactly zero on the boundaries because of the hard constraints. The error in α is larger than in c because α affects the solution more weakly, and the L1 discretisation error can be absorbed as a small shift in α . In the noise study, the boundary and initial data were noise-free, which helps the recovery of c .

6. Conclusions

PINNs solved both forward and inverse mechanics problems accurately without a mesh, using only the governing equations and sparse, noisy data. Recovering a parameter needed only a small change to the forward code. Extending the standard PINN with a numerical L1 operator (fPINN) allowed it to handle fractional derivatives, which autograd alone cannot compute, and hard constraints removed the boundary errors observed in the reference work. Limitations include long training compared with classical solvers for simple problems, sensitivity to loss weights, and an accuracy floor set by the fractional discretisation. Future work: coupled multi-physics problems (poroelasticity), adaptive sampling of collocation points, and Δ -PINNs for complex geometries.